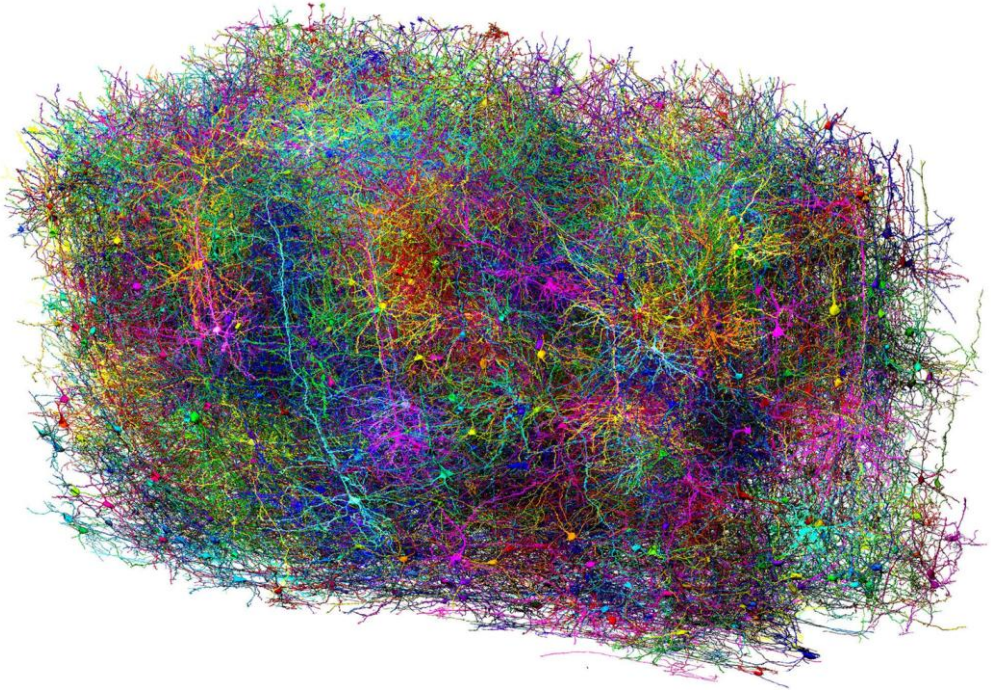
Learning in neural networks

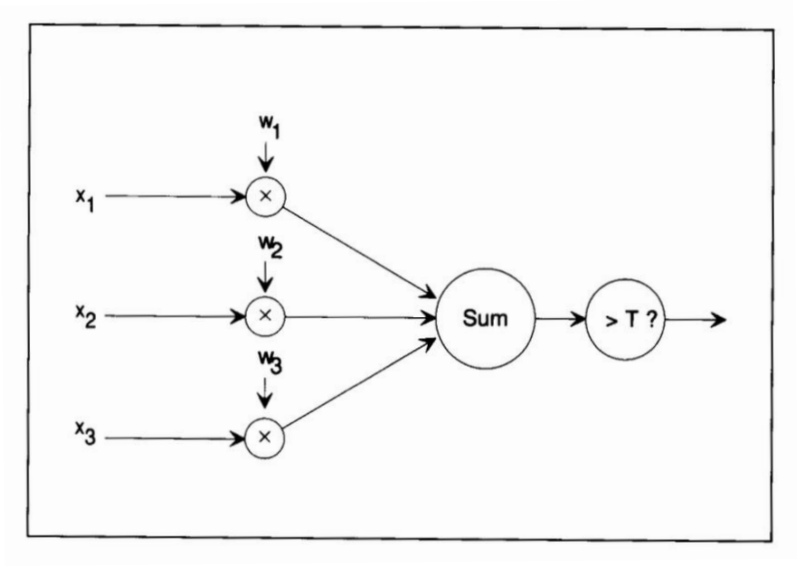
How can artificial neurons automatically learn patterns?



Credit: Forrest Collman/Allen Institute

How to learn the decision boundary automatically?

We need an algorithm for that: enter the *Perceptron* *(Rosenblatt, 1957)*



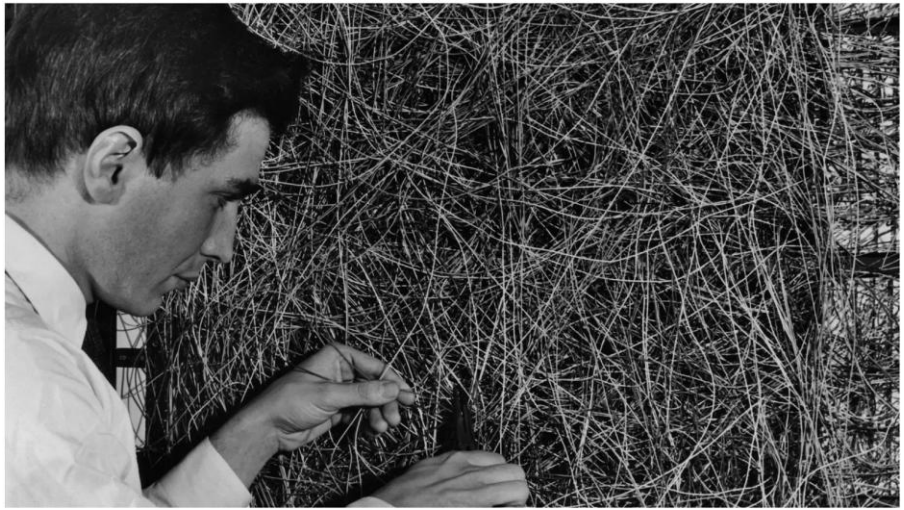
Credit: Winston, Artificial Intelligence

Wikipedia

Frank Rosenblatt

The Perceptron Convergence Procedure Guarantees Success Whenever Success Is Possible

Remarkably, there is a procedure that discovers a successful set of weights for a perceptron given that a successful set of weights exists. Moreover, the procedure is incredibly straightforward. Basically, you start with all weights initially set to 0. Then, you try your perceptron on all samples, one at a time. Whenever your perceptron makes a mistake, you change the weights so as to make the mistake less likely; otherwise, you do nothing.



Un hombre trabaja en el Perceptrón de Frank Rosenblatt FREDERIC LEWIS/ARCHIVE PHOTOS/GETTY IMAGES

https://es.wired.com/articulos/conoce-a-frank-rosenblatt-creador-de-la-primera-inteligencia-artificial

---

To train a perceptron,

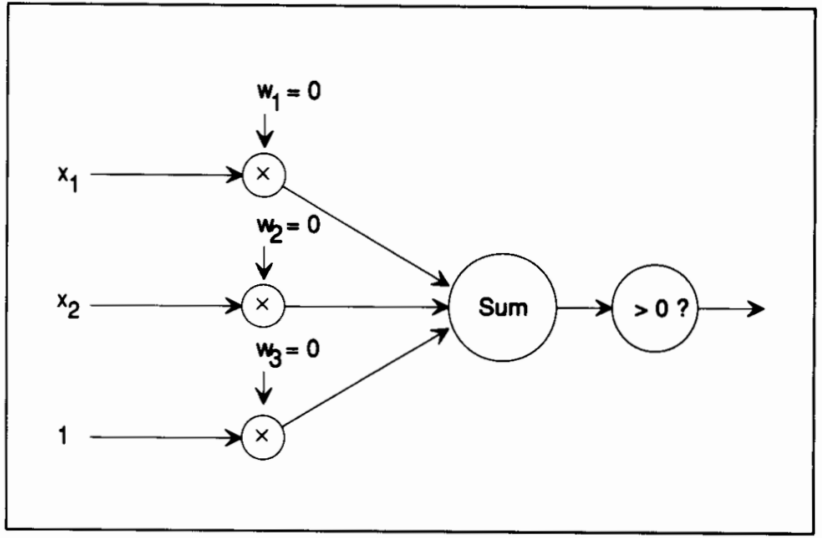
- ▷ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▷ If the perceptron yields the wrong answer,
    - ▷ If the perceptron says no when it should say yes, add the input vector **x** to the weight vector.
    - ▷ Otherwise, subtract the input vector **x** from the weight vector.
  - ▷ Otherwise, do nothing.

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR

Figure 23.4 A straight-through perceptron with two ordinary inputs and a threshold-replacing input. This one is ready to be trained to recognize the logical Or of its inputs. Note that the always-1 input and corresponding weight take the place of a nonzero threshold.

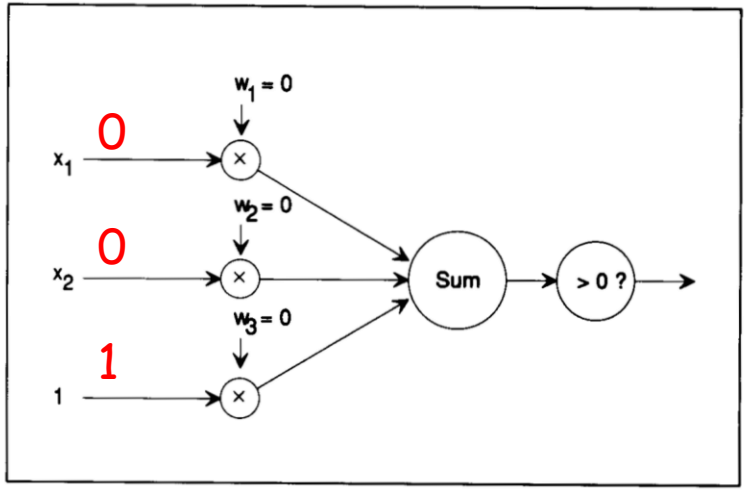


Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

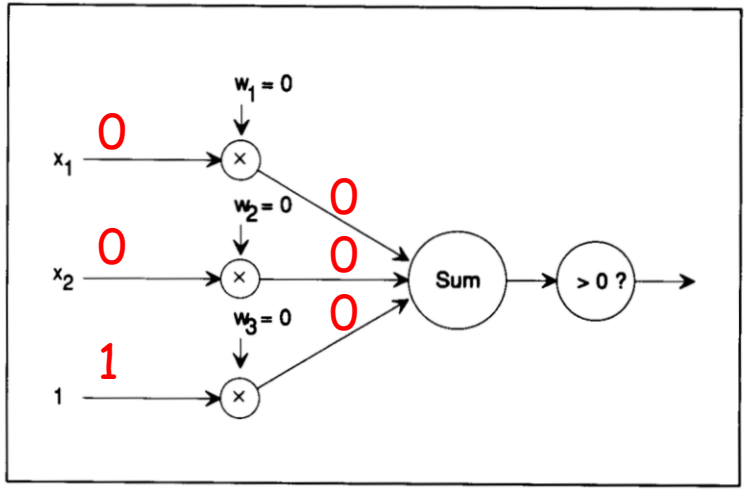
To train a perceptron,

▹ Until the perceptron yields the correct result for each training sample, for each sample,

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

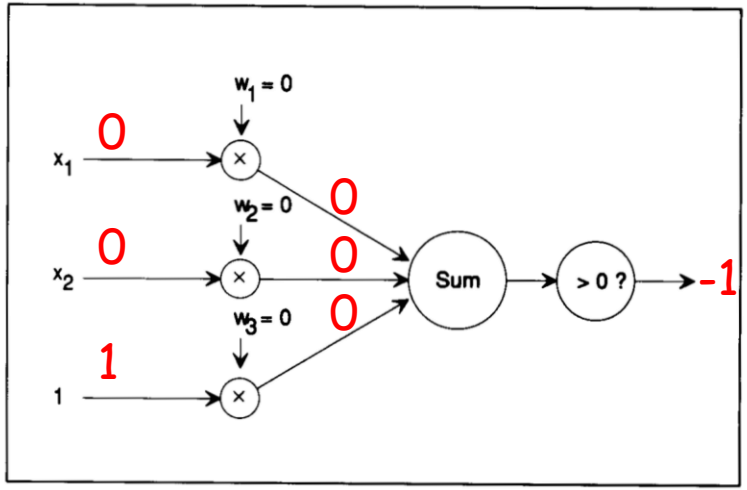
To train a perceptron,

▷ Until the perceptron yields the correct result for each training sample, for each sample,

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

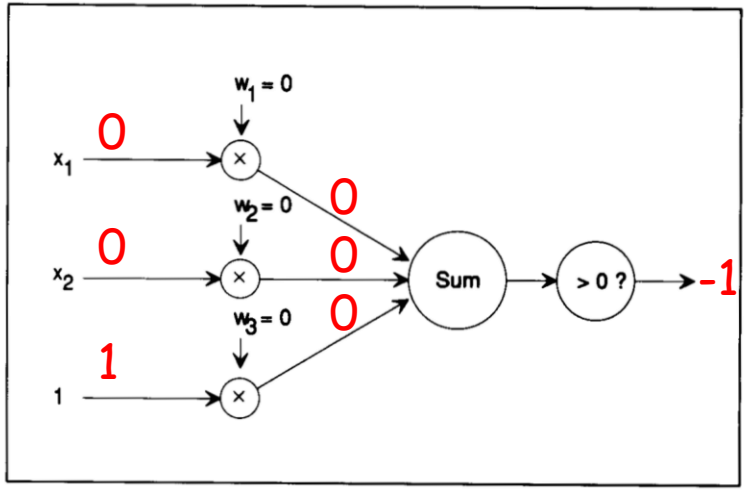
To train a perceptron,

▷ Until the perceptron yields the correct result for each training sample, for each sample,

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

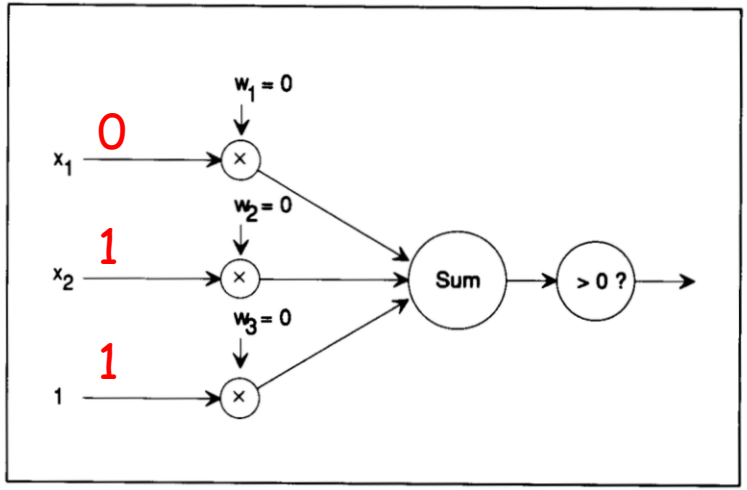
To train a perceptron,

- ▷ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▷ ~~If the perceptron yields the wrong answer,~~
  - ▷ Otherwise, do nothing.

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

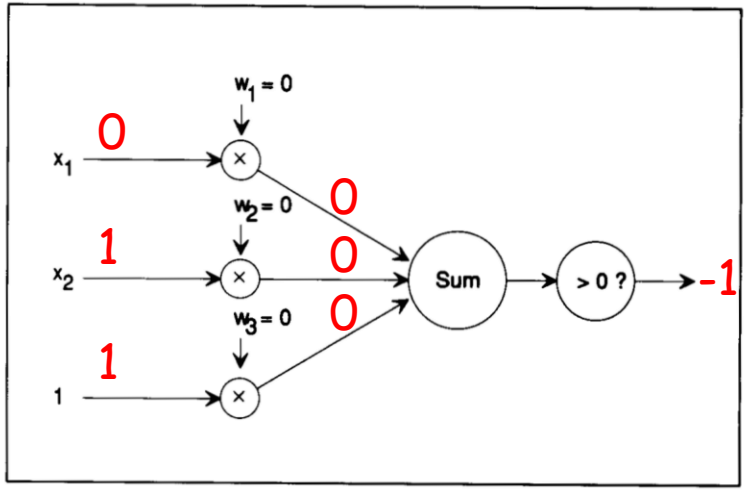
To train a perceptron,

▷ Until the perceptron yields the correct result for each training sample, for each sample,

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

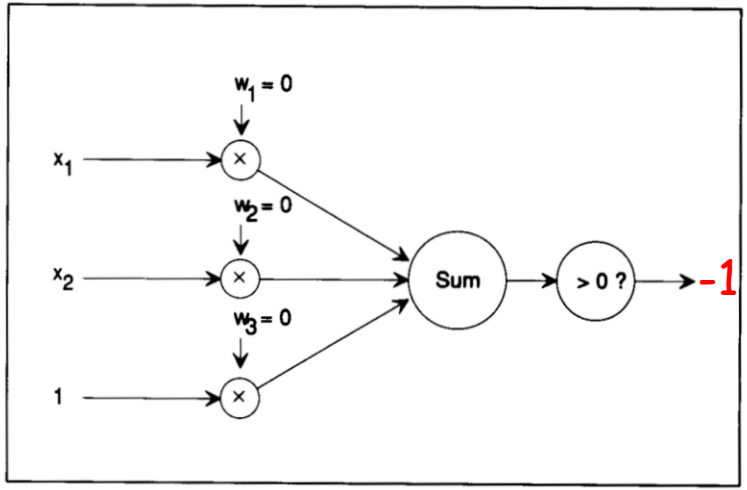
To train a perceptron,

- ▷ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▷ If the perceptron yields the wrong answer,
    - ▷ If the perceptron says no when it should say yes, add the input vector **x** to the weight vector.

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

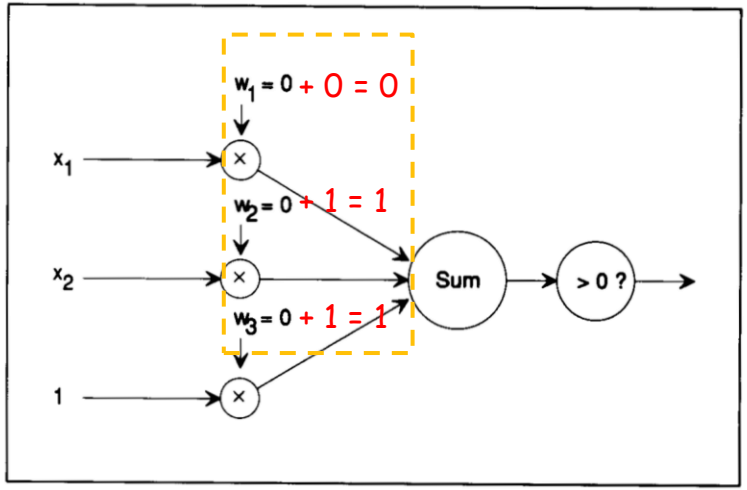
To train a perceptron,

- ▷ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▷ If the perceptron yields the wrong answer,
    - ▷ If the perceptron says no when it should say yes, add the input vector **x** to the weight vector.

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

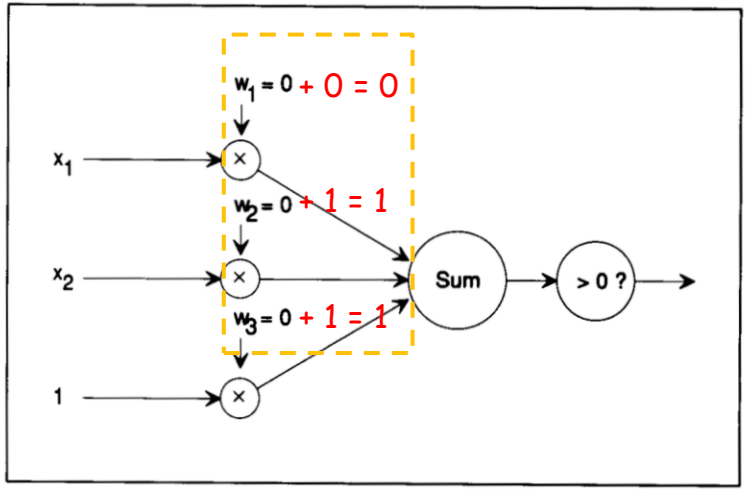
To train a perceptron,

- ▹ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▹ If the perceptron yields the wrong answer,
    - ▹ If the perceptron says no when it should say yes, add the input vector **x** to the weight vector.

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR

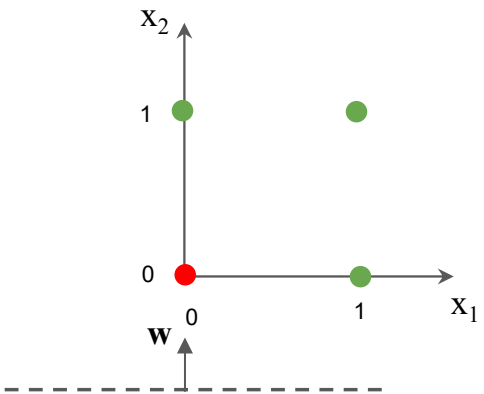


Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

To train a perceptron,

- ▹ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▹ If the perceptron yields the wrong answer,
    - ▹ If the perceptron says no when it should say yes, add the input vector **x** to the weight vector.

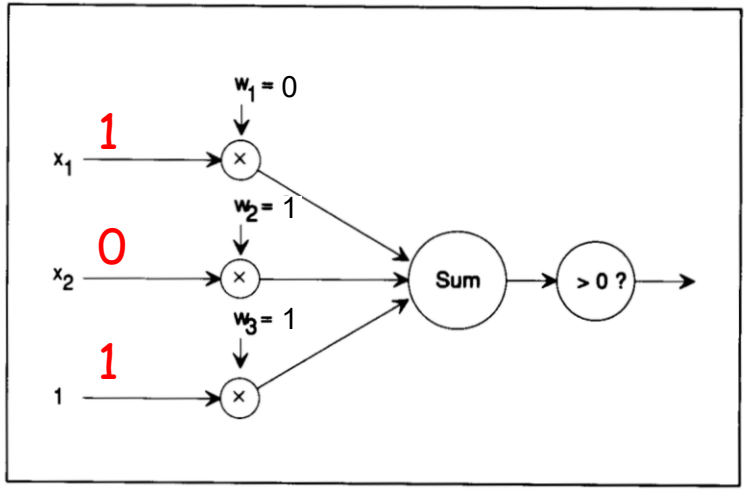
Geometrically:



Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

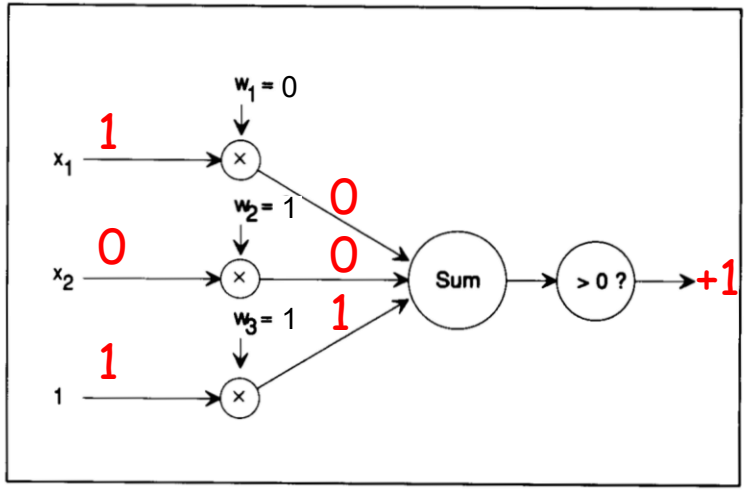
To train a perceptron,

▷ Until the perceptron yields the correct result for each training sample, for each sample,

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

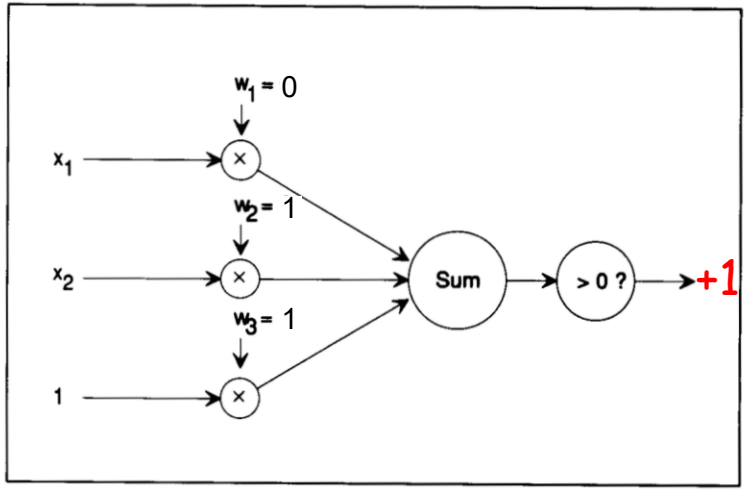
To train a perceptron,

▷ Until the perceptron yields the correct result for each training sample, for each sample,

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

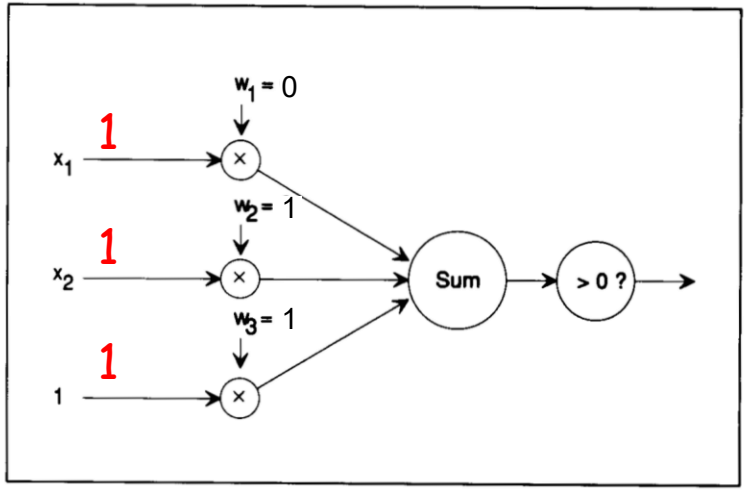
To train a perceptron,

- ▷ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▷ ~~If the perceptron yields the wrong answer,~~
  - ▷ Otherwise, do nothing.

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

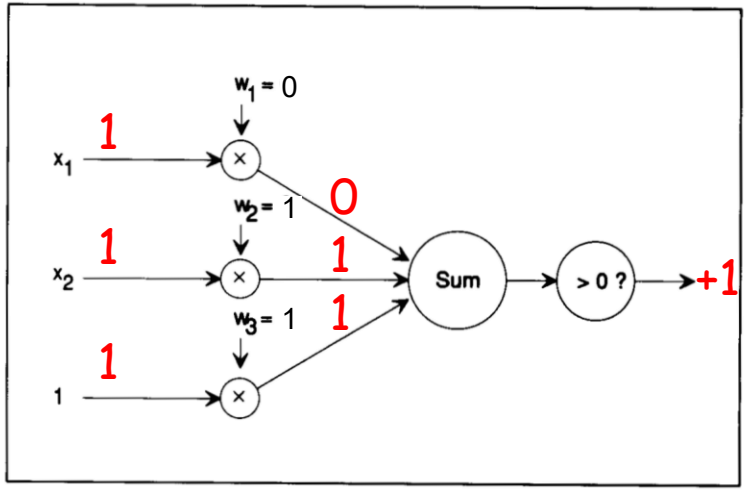
To train a perceptron,

▷ Until the perceptron yields the correct result for each training sample, for each sample,

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

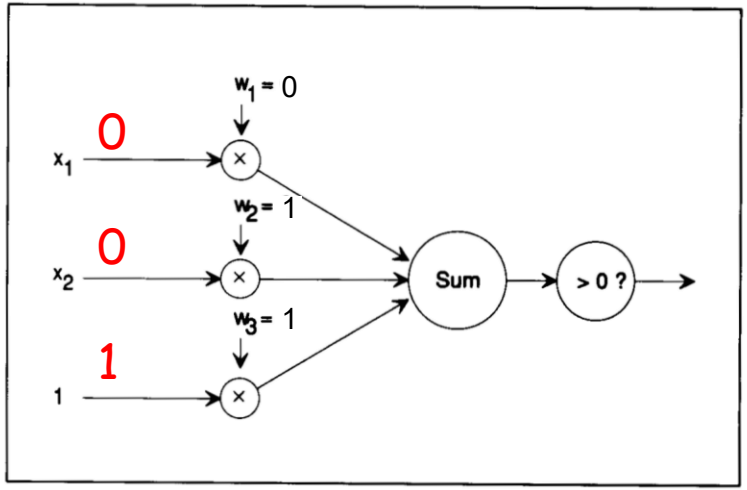
To train a perceptron,

- ▷ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▷ ~~If the perceptron yields the wrong answer,~~
  - ▷ Otherwise, do nothing.

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

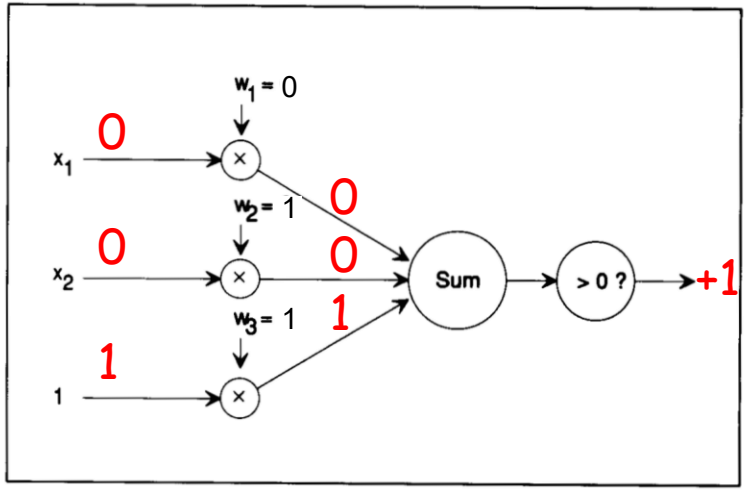
To train a perceptron,

▷ Until the perceptron yields the correct result for each training sample, for each sample,

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

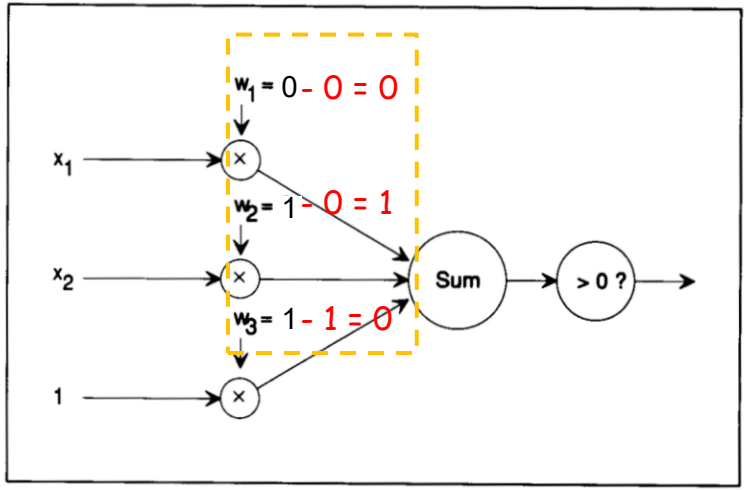
To train a perceptron,

- ▹ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▹ If the perceptron yields the wrong answer,
    - ▹ If the perceptron says no when it should say yes, add the input vector **x** to the weight vector.
    - ▹ Otherwise, subtract the the weight vector.

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

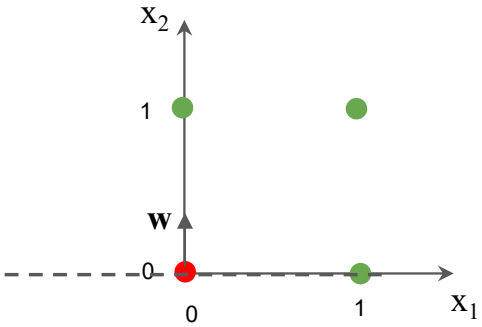
Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

To train a perceptron,

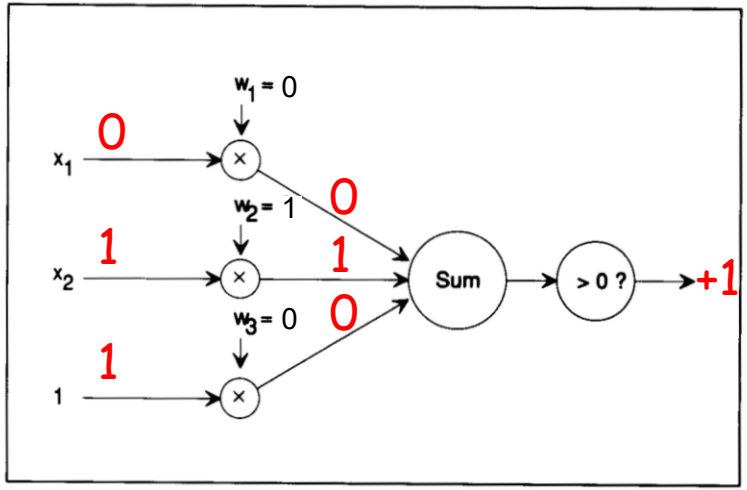
- ▹ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▹ If the perceptron yields the wrong answer,
    - ▹ If the perceptron says no when it should say yes, add the input vector **x** to the weight vector.
    - ▹ Otherwise, subtract the input vector **x** from the weight vector.



Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

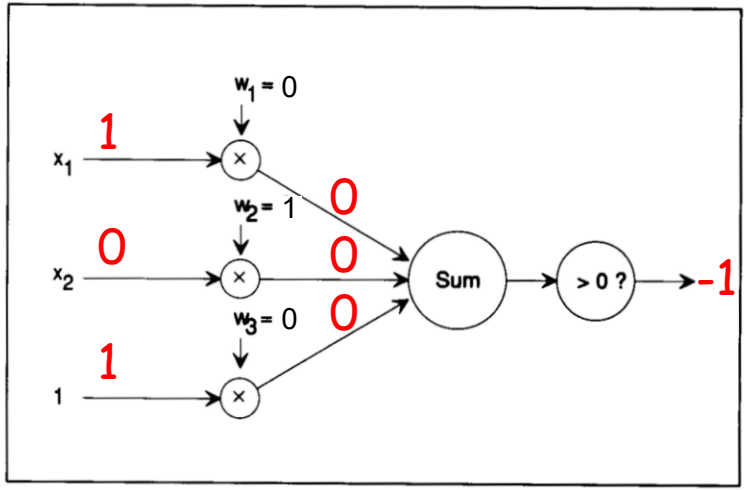
To train a perceptron,

- ▹ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▹ If the perceptron yields the wrong answer,
    - ▹ Otherwise, do nothing.

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

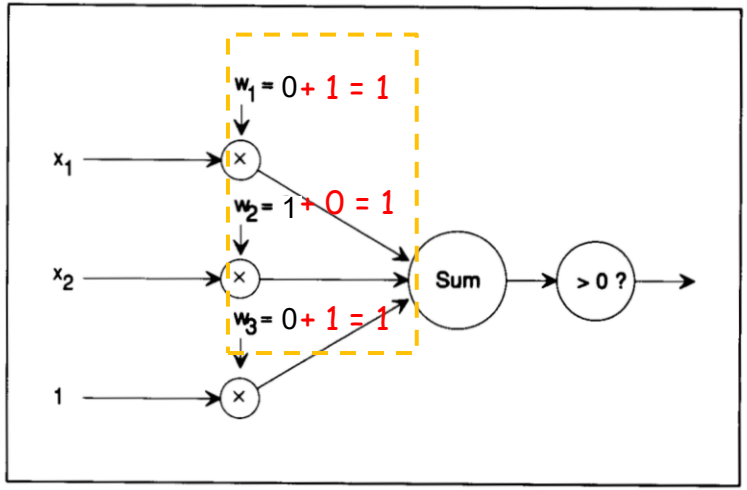
To train a perceptron,

- ▷ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▷ If the perceptron yields the wrong answer,
    - ▷ If the perceptron says no when it should say yes, add the input vector **x** to the weight vector.

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

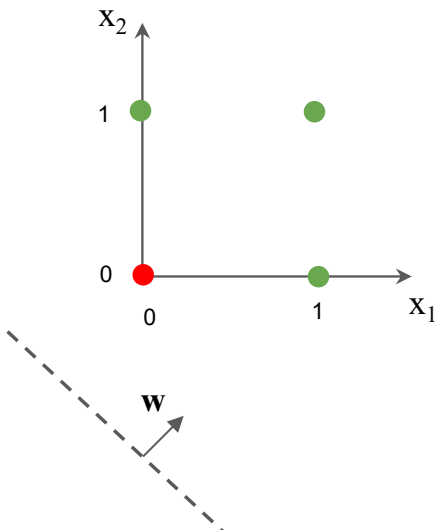
Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

To train a perceptron,

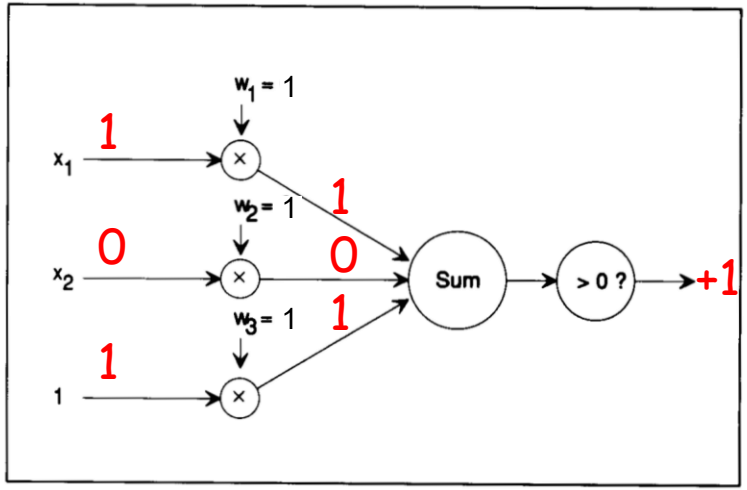
- ▹ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▹ If the perceptron yields the wrong answer,
    - ▹ If the perceptron says no when it should say yes, add the input vector **x** to the weight vector.



Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

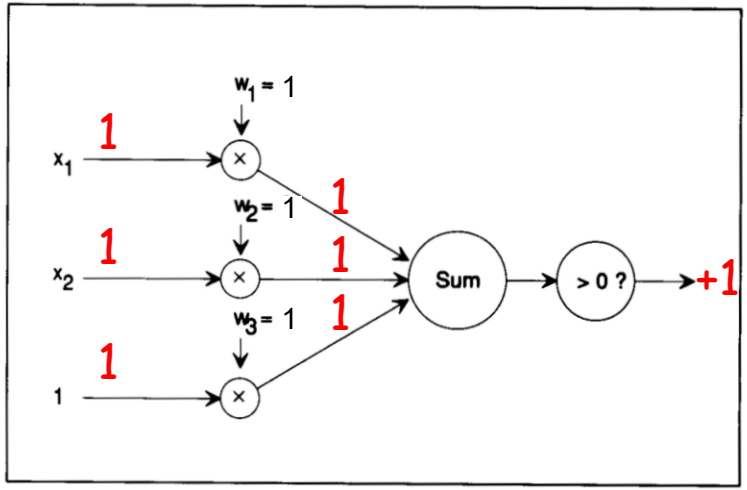
To train a perceptron,

- ▹ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▹ If the perceptron yields the wrong answer,
    - ▹ Otherwise, do nothing.

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

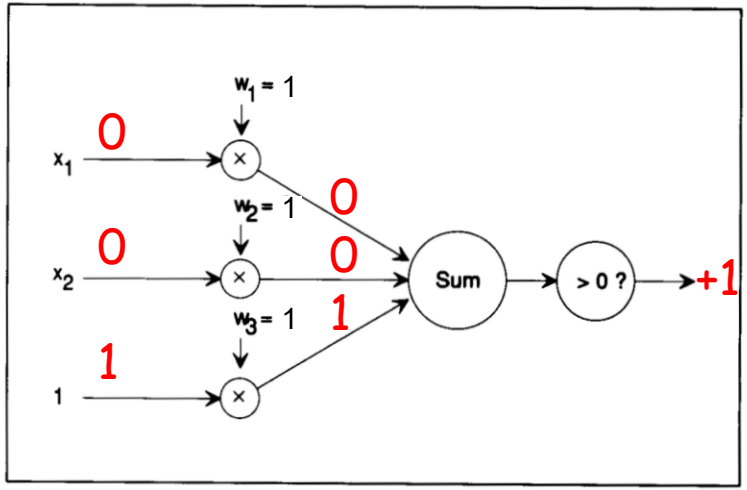
To train a perceptron,

- ▷ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▷ If the perceptron yields the wrong answer,
    - ▷ Otherwise, do nothing.

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

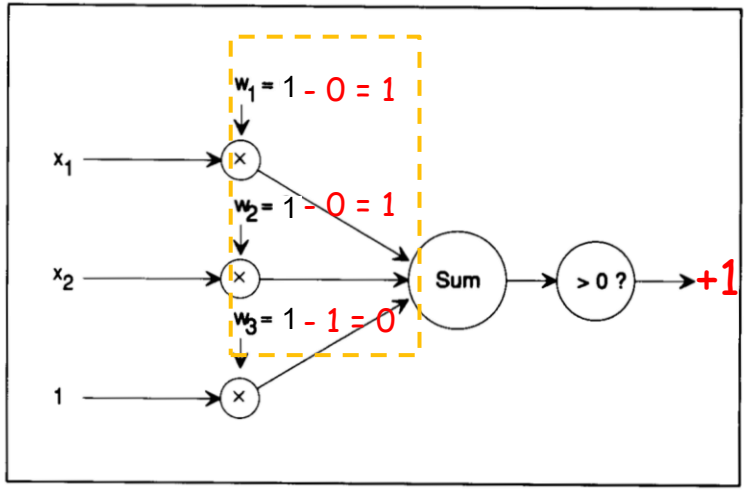
To train a perceptron,

- ▷ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▷ If the perceptron yields the wrong answer,
    - ▷ If the perceptron says no when it should say yes, add the input vector **x** to the weight vector.
    - ▷ Otherwise, subtract the input vector **x** from the weight vector.

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR



Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

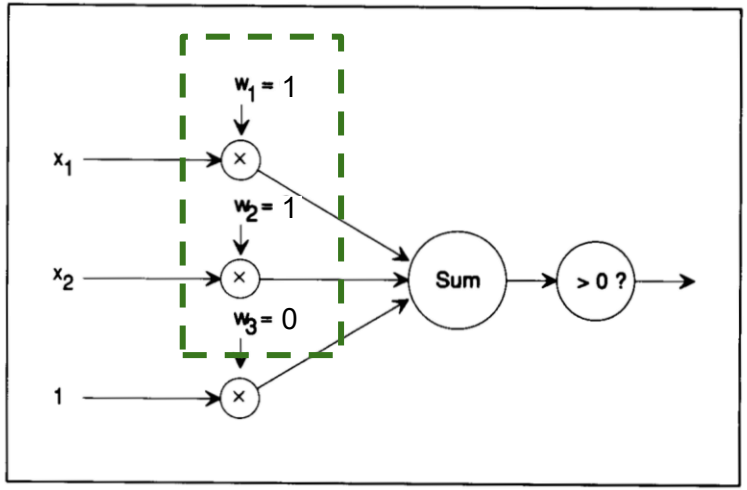
To train a perceptron,

- ▷ Until the perceptron yields the correct result for each training sample, for each sample,
  - ▷ If the perceptron yields the wrong answer,
    - ▷ If the perceptron says no when it should say yes, add the input vector **x** to the weight vector.
    - ▷ Otherwise, subtract the input vector **x** from the weight vector.

Adapted from: Winston, Artificial Intelligence

The *perceptron* in action

Example: logical OR

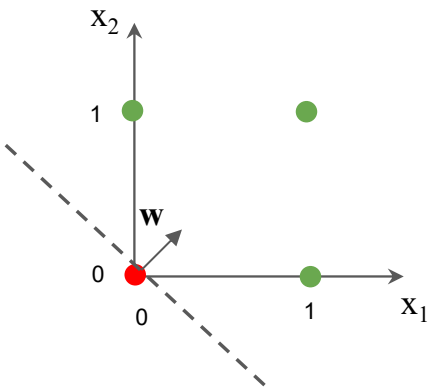


Sample	x_1	x_2	x_3	Desired output
1	0	0	1	-1
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

To train a perceptron,

▷ Until the perceptron yields the correct result for each training sample, ~~for each sample,~~

Done!



Adapted from: Winston, Artificial Intelligence

The perceptron convergence procedure is amazing

- On the surface, it is strange that the perceptron convergence procedure ever succeeds.

- You would think that, each time it makes a change to improve the handling of one particular sample, that change could ruin what is done to other samples.

- And yet, it is not hard to prove mathematically that it will always converge **if the data are linearly separable**!

More info here (optional material for this course)

Adapted from: Winston, Artificial Intelligence

https://havi-muro.github.io/ML_demos/learning

Perceptron Learning Rule — Upper-Right Quadrant (Random + More Scatter, Auto-Fit)

Separated but more scattered clusters (try bias b = 0). Click **Train** t

How to use this demo

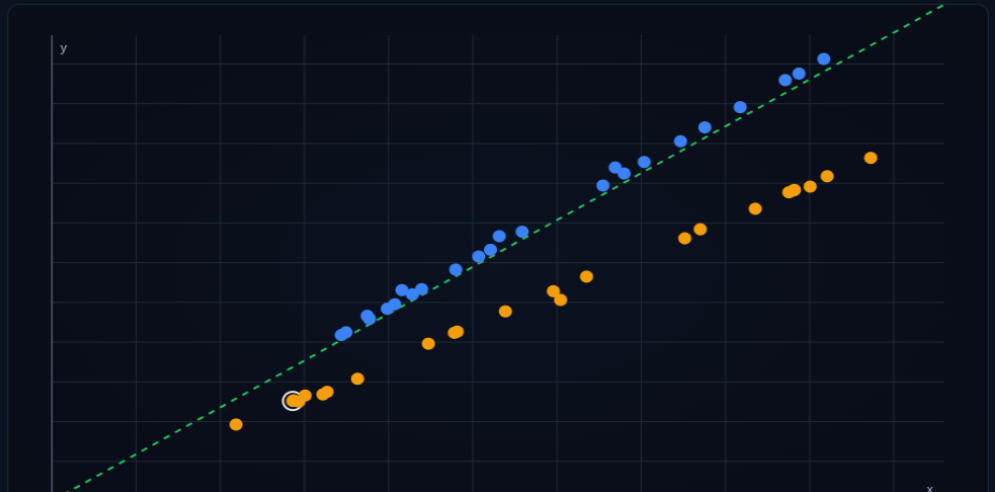
1. Choose the *learning rate* (η) and the fixed *bias* (b). For this dataset, `b = 0` works well.
2. Click **Train**. Each click picks *one random observation* (blue or orange) and applies the perceptron update `w ← w + η · y · x` repeatedly until that point is correctly classified.
3. If the selected observation is *already correctly classified*, you'll see that **weights don't change**.
4. Repeat until the status shows that all points are perfectly separated.

What you'll observe/learn

- The dashed decision boundary $w_x x + w_y y + b = 0$ rotates/shifts as `w` changes.
- The green weight vector is the boundary's normal; its direction encodes the separator's orientation.
- Higher η converges in fewer presses (bigger jumps); lower η is smoother but slower.
- With `b=0`, the separator passes through the origin. The dataset remains linearly separable.

Q: what happens if a perfect separator does not exists?

Dataset & Decision Boundary



Controls

Learning rate (η): 0.01

Bias (b) — fixed during training: 0.0

Train | Reset | Randomize data

All points are perfectly separated 🎉. Press **Randomize data** to try a new dataset or **Reset** to start over.

Weights

`w = (-0.01, 0.01)`

`‖w‖ = 0.01`

Accuracy

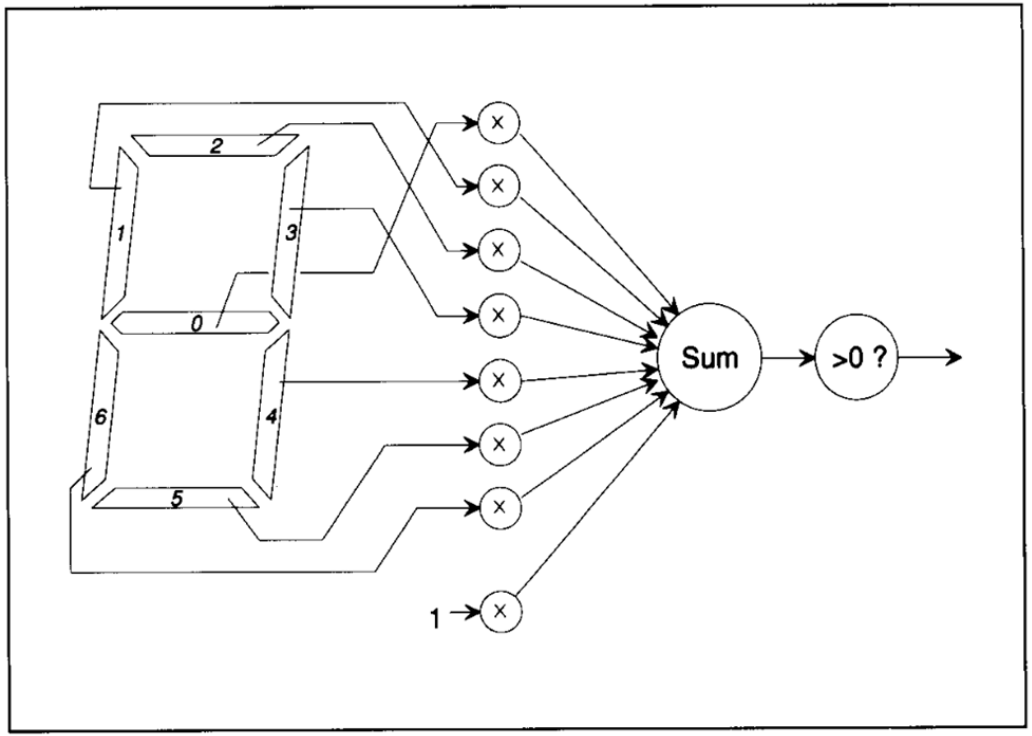
`Accuracy = 100.00%`

`Misclassified = 0 / 48`

What else can perceptrons do?

Perceptrons can learn to identify individual digits:

Figure 23.7 A digit-recognizing perceptron. Seven of the inputs are attached to the seven line segments; the eighth is attached to an always-1 input, thus replacing a threshold.



Adapted from: Winston, Artificial Intelligence

What else can perceptrons do?

Perceptrons can learn to identify images:

What we see

0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

What the computer 'sees'

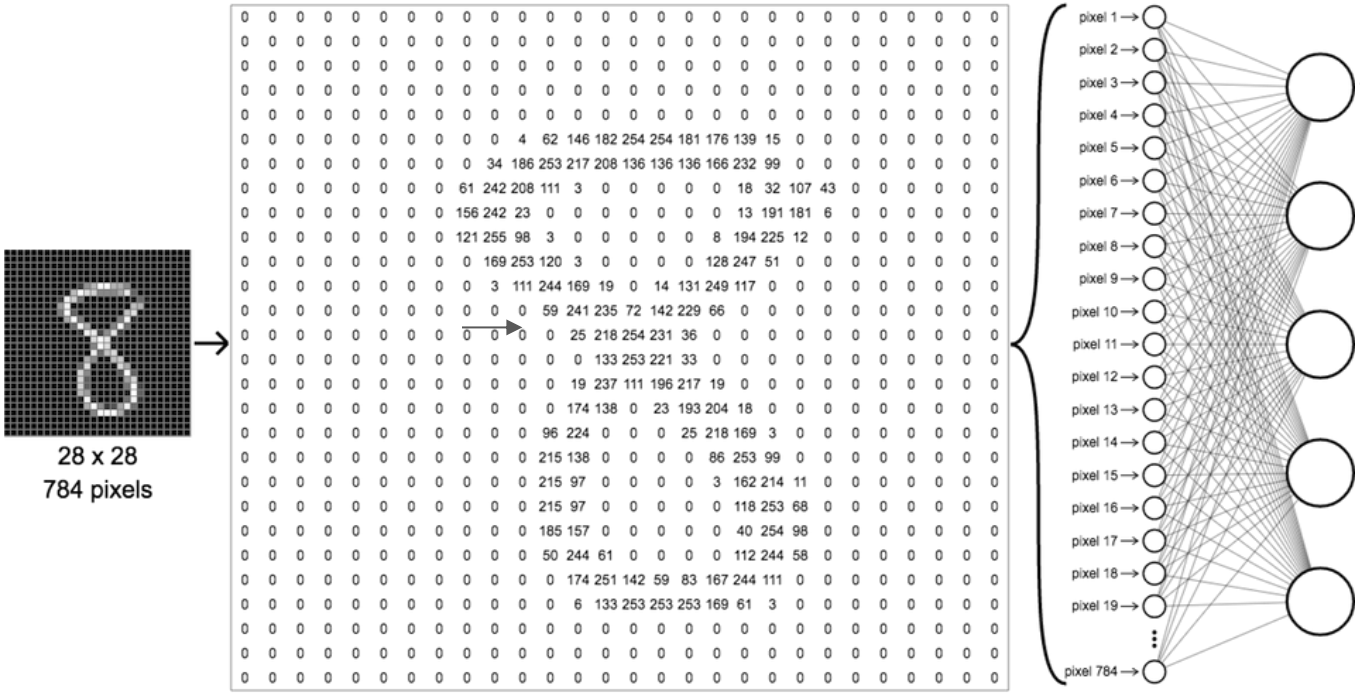
0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

numbers between [0, 255] represent intensity

Adapted from: Winston, Artificial Intelligence

What else can perceptrons do?

E.g.: perceptrons can learn to identify simple images

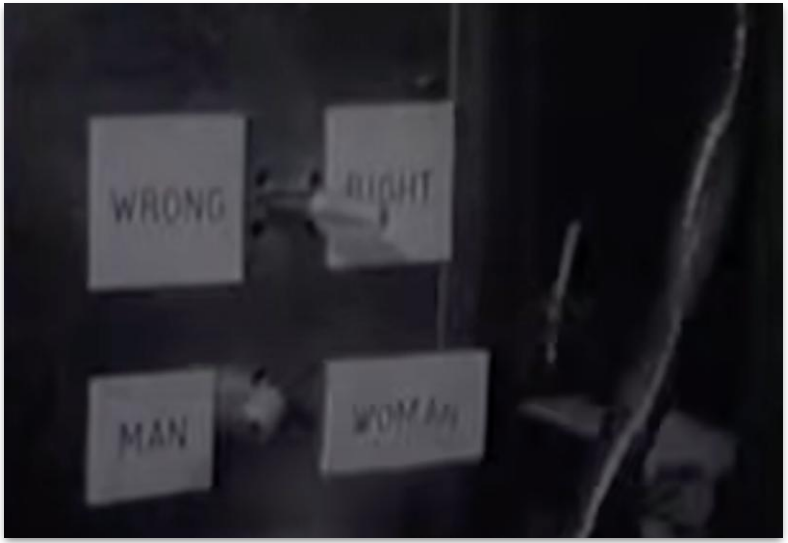


Is this an "8"?

Perceptrons can only identify 1 class vs. others, so each digit would need its own neuron

What else can perceptrons do?

Perceptrons can (sometimes) even learn to classify complex images:

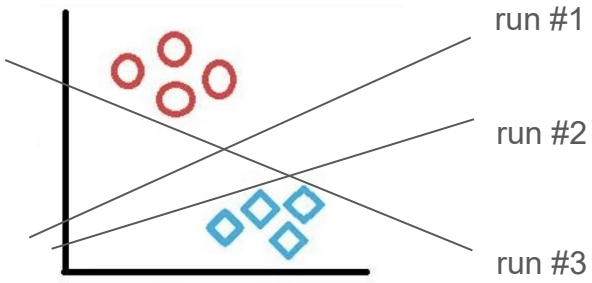


What needs to be true for this to succeed?

Link: [Perceptron Research from the 50's & 60's, clip]

But… often its solution won't adapt well to more samples

Important note:
Depending on the order in which the algorithm traverses the training samples, it might reach different solutions that correctly separate the data!



Some are "better" than others… how?

No notion of "optimality" of a solution

But… there are simple tasks that perceptrons cannot do

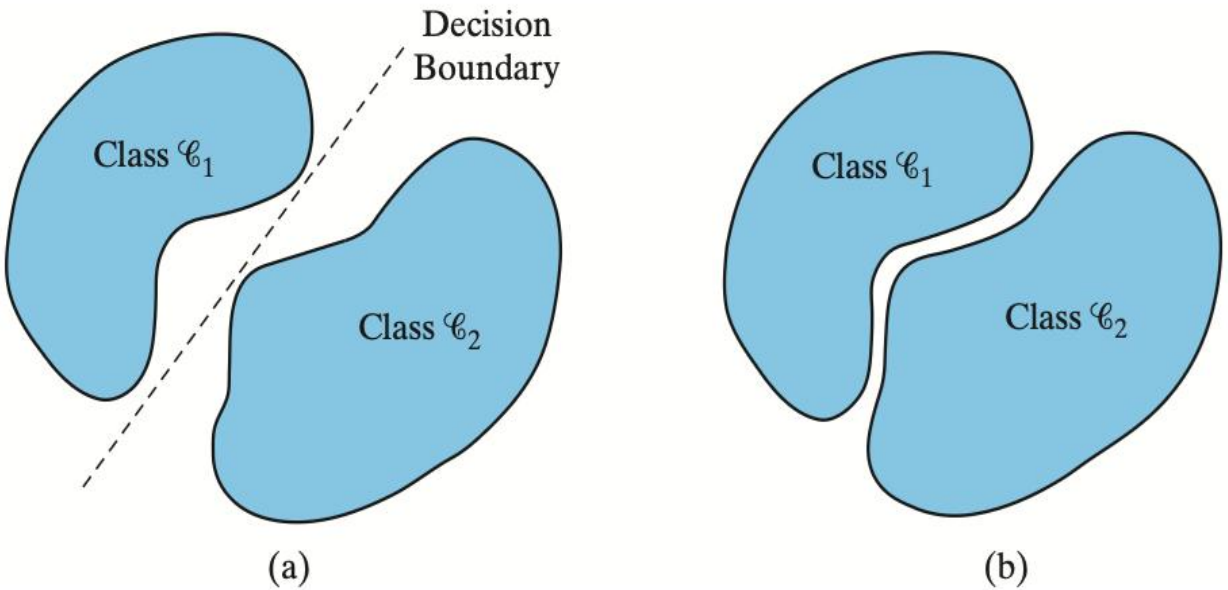
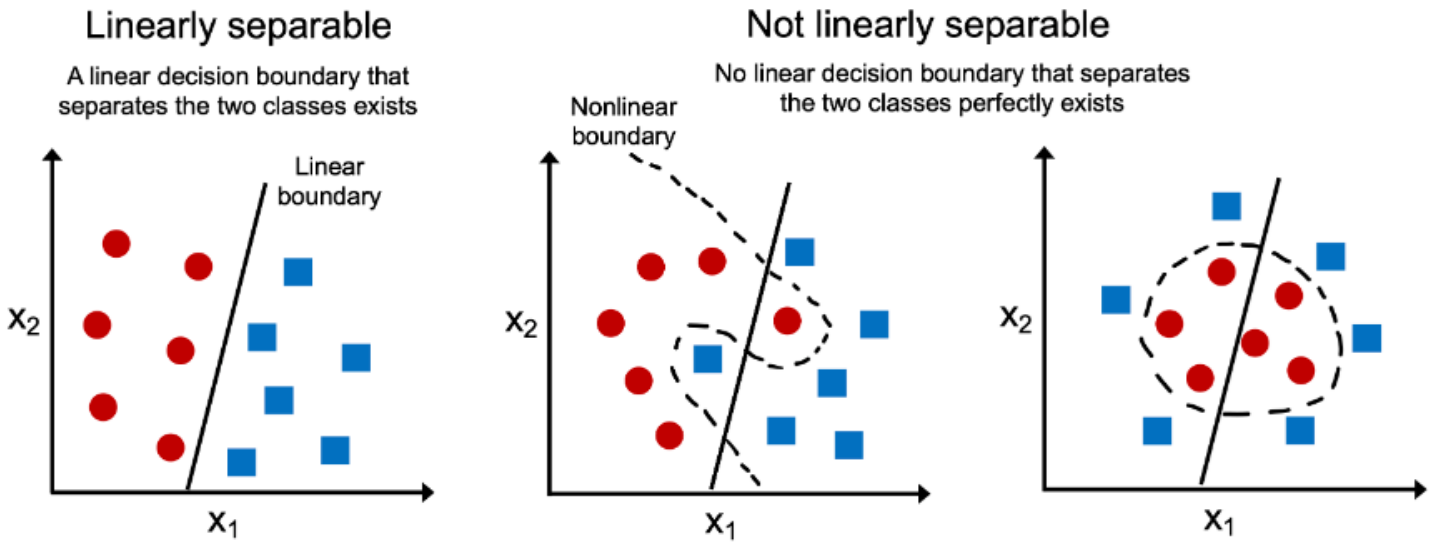


FIGURE 1.4 (a) A pair of linearly separable patterns. (b) A pair of non-linearly separable patterns.

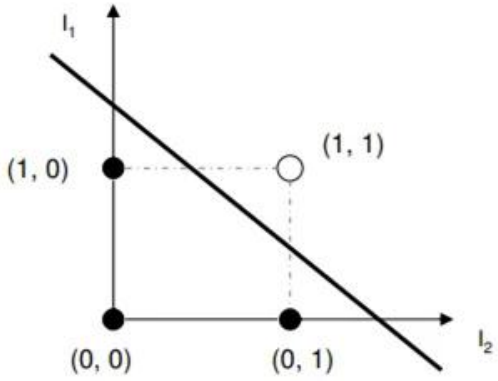
But… there are simple tasks that perceptrons cannot do



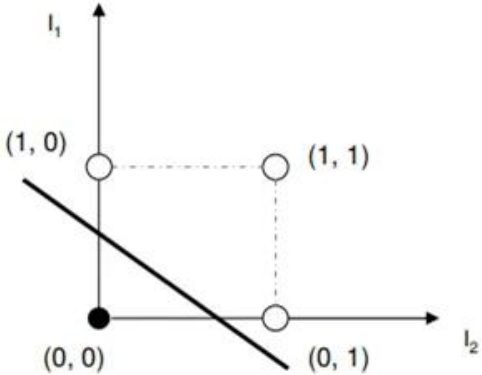
Even a single point can prevent the perceptron from ever reaching a solution!

But… there are simple tasks that perceptrons cannot do

AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1



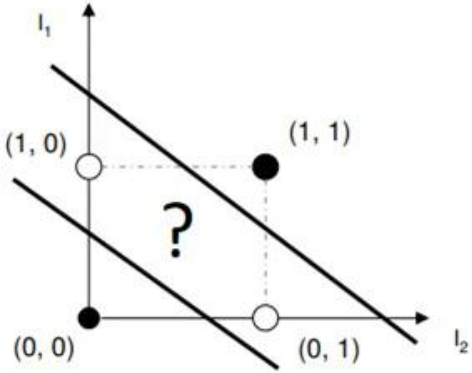
OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1



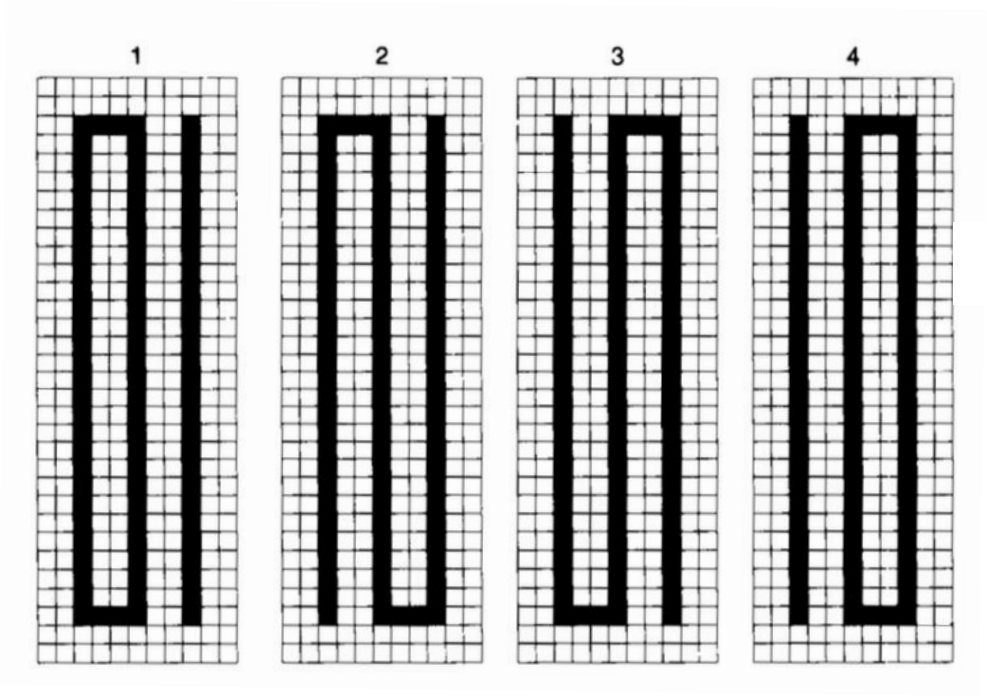
The exclusive-or (XOR) operator:

"one or the other, but not both"

XOR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	0



But… there are simple tasks that perceptrons cannot do

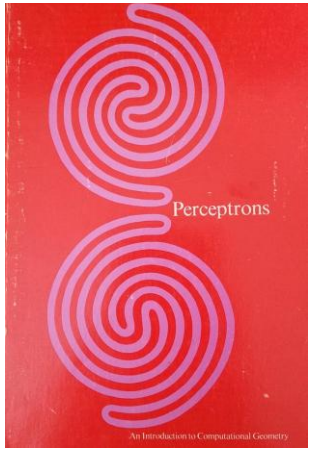


Which patterns are connected vs. disconnected?

Minsky and Papert (1969) wrote an influential book demonstrating Perceptron's representational limitations

- In vision, if patterns are represented with local features, can't represent symmetry, connectivity

https://rodsmith.nz/wp-content/uploads/Minsky-and-Papert-Perceptrons.pdf



Minsky & Papert, *Perceptrons*

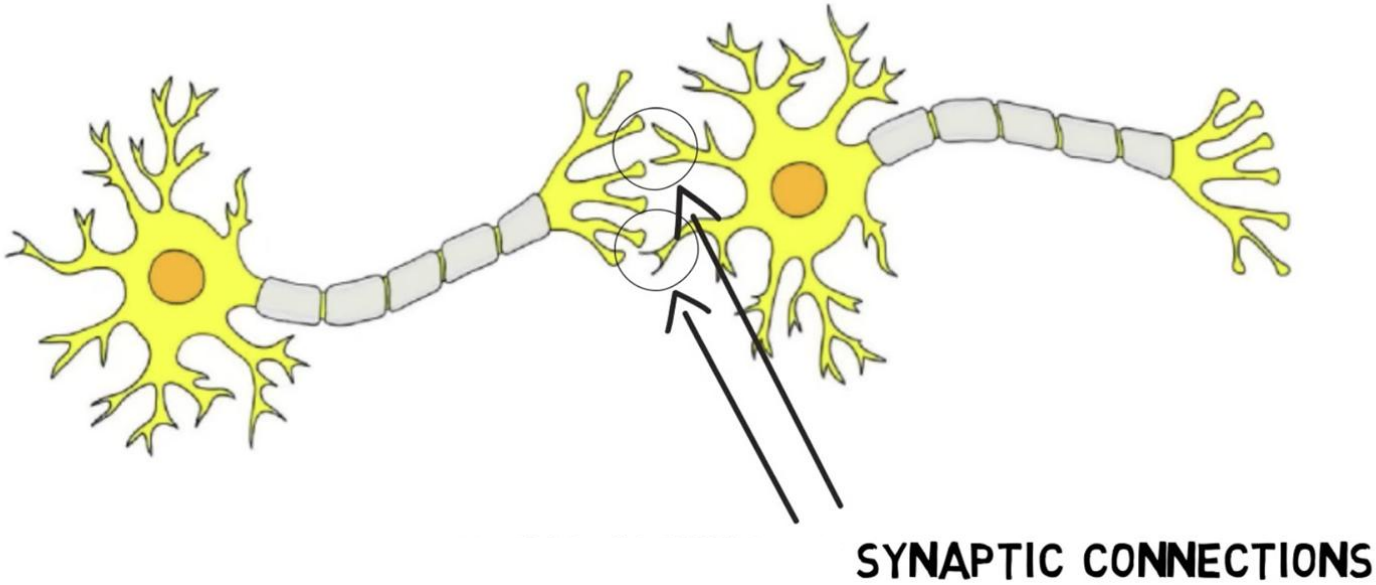
Adapted from: Winston, AI & V. Manfredi (Wesleyan)

Biological neural networks

How do *our own neurons* learn?

How do *our own neurons* learn?

- The changing and shaping of connections in our brain is known as **synaptic plasticity**
- To understand how this happens, let's dig into how biological neurons communicate.



Credit: https://www.youtube.com/watch?v=-mHgPfXHzJE, https://www.youtube.com/watch?v=KM3xUxkj-3o

How do *our own neurons* learn?

- Neurons communicate using both electrical and chemical signals.
- Life experiences change the nervous system.

BrainFacts.org

Spike timing-dependent plasticity

- **STDP** = a framework for understanding plasticity one postsynaptic spike at a time

Credit: L. White (Duke, coursera.org)

What happens when the firing of a neuron A leads to the firing of a neuron B:

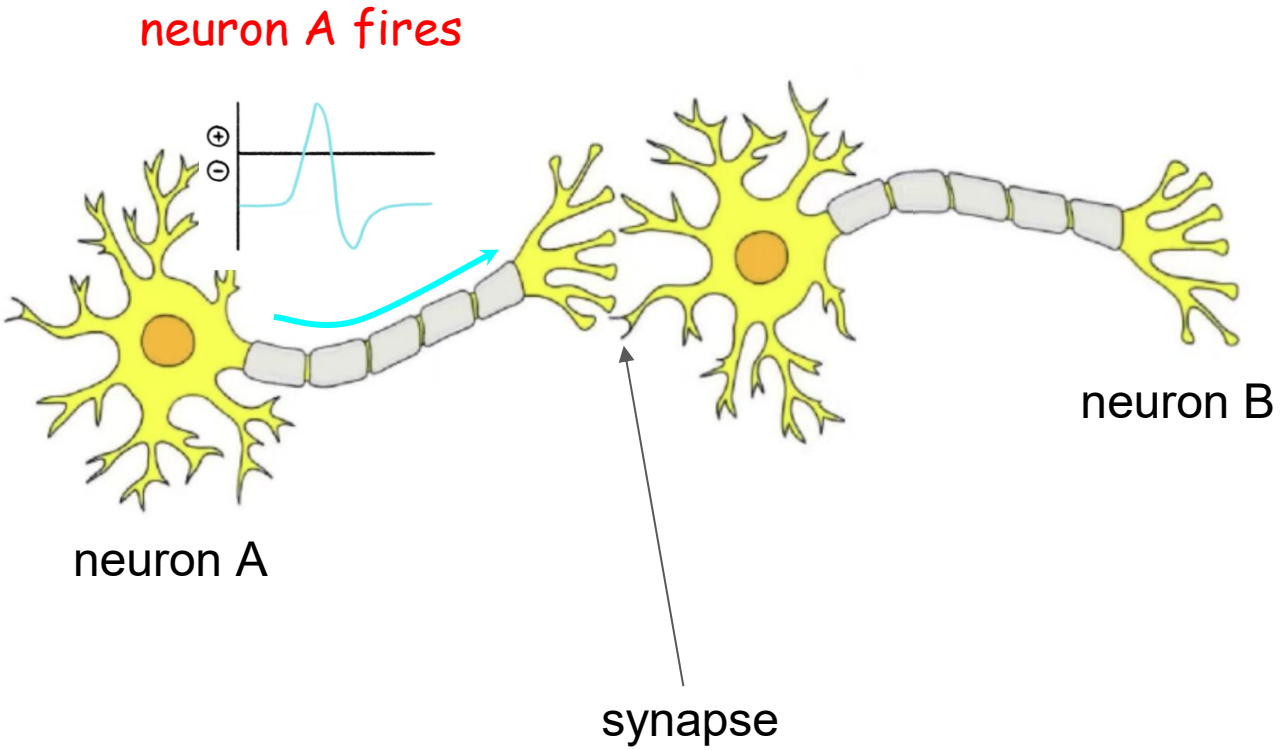


Image credit: https://www.youtube.com/watch?v=-mHgPfXHzJE

What happens when the firing of a neuron A leads to the firing of a neuron B:

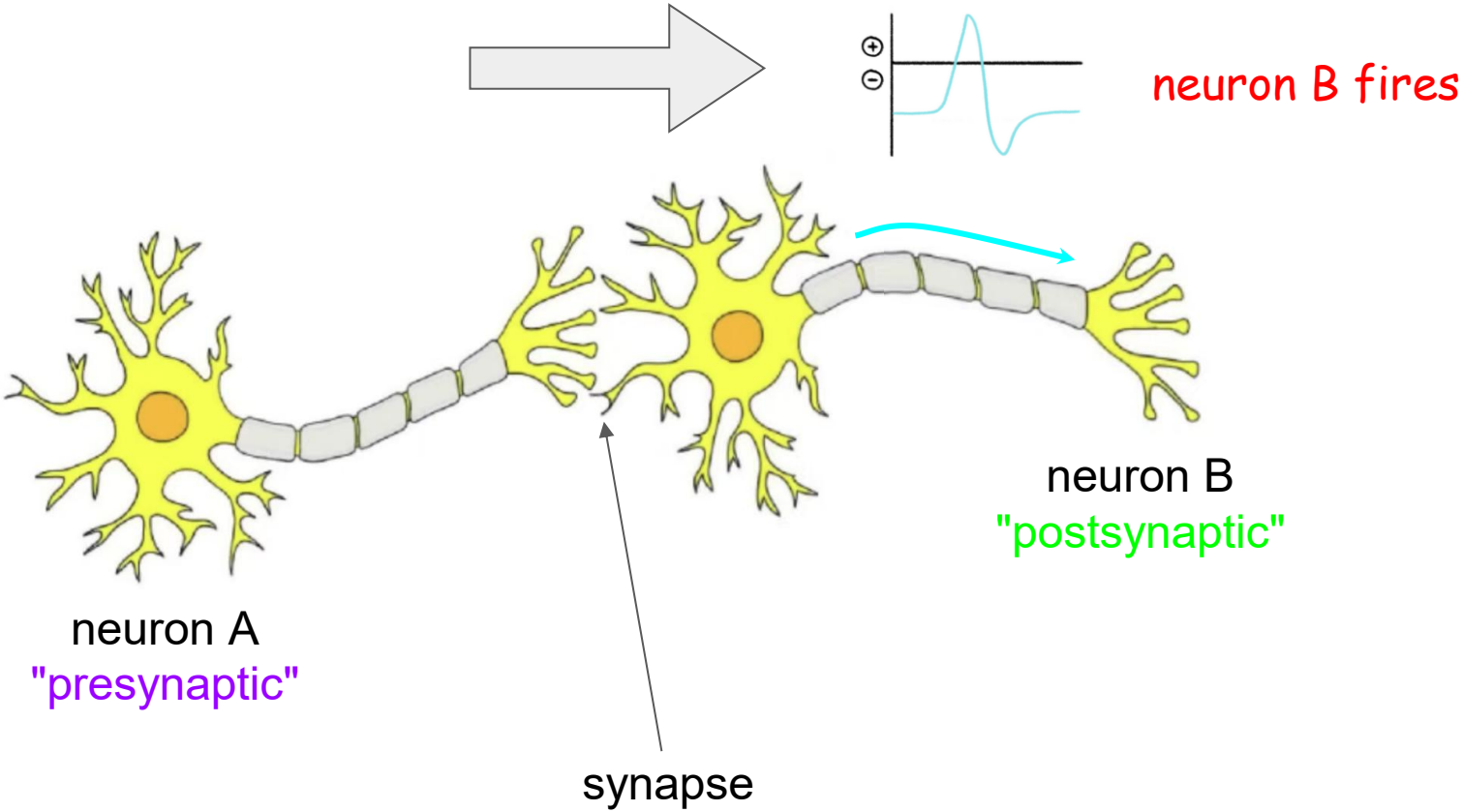
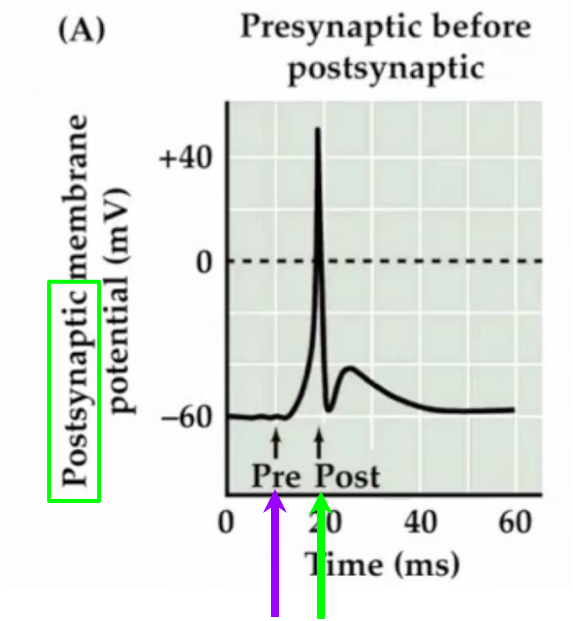


Image credit: https://www.youtube.com/watch?v=-mHgPfXHzJE

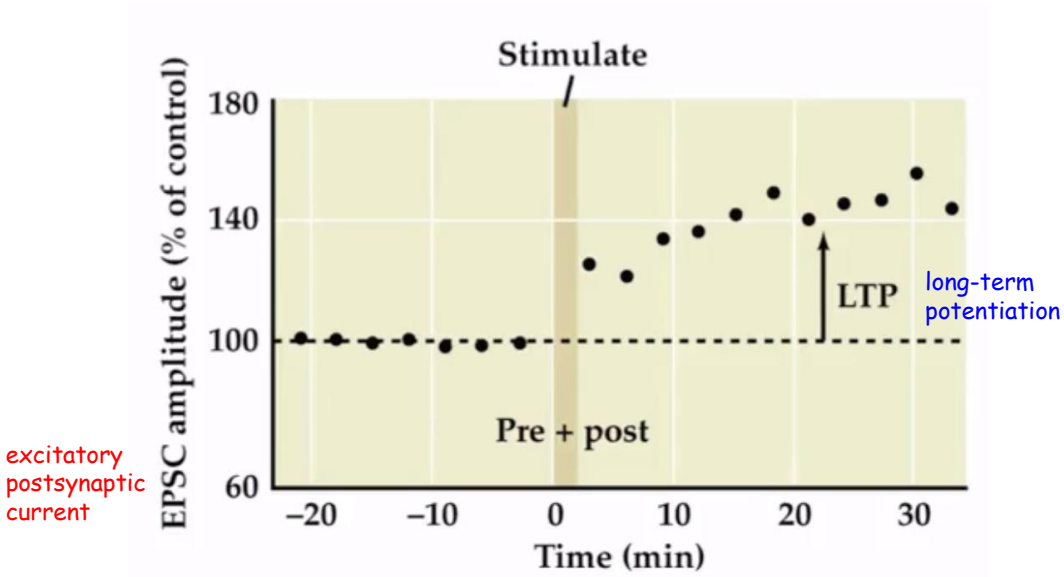
Scenario 1: neuron B consistently fires just after neuron A

in other words, a spike in A contributes directly to a spike in B



action potential in the presynaptic cell (A)

action potential in the postsynaptic cell (B)



excitatory postsynaptic current

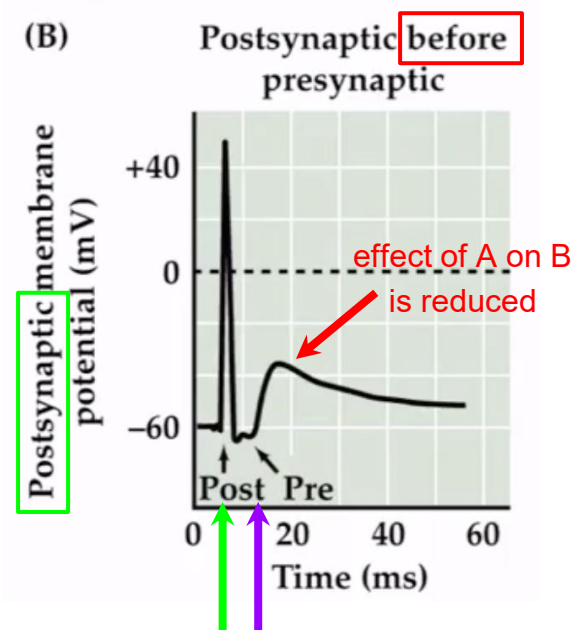
Amplitude of the postsynaptic current (strength of the connection): Following some structured stimulation period ("training") where neuron B consistently fires after A, an increase of about 40% is observed.

NEUROSCIENCE 5e, Figure 8.18 (Part 1)

© 2012 Sinauer Associates, Inc.

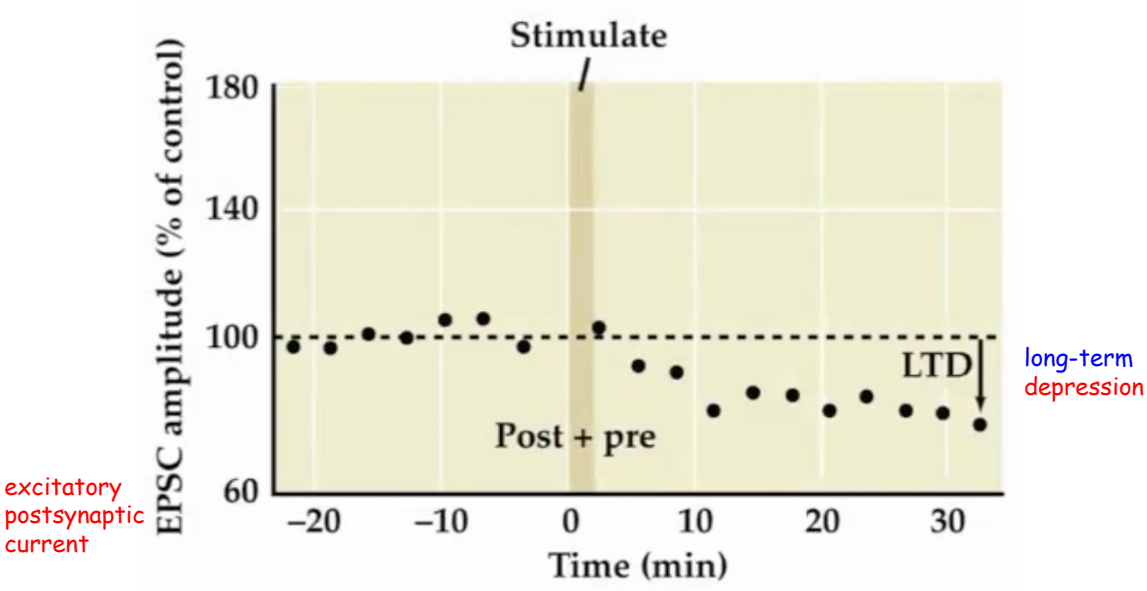
Credit: L. White (Duke, coursera.org)

Scenario 2: neuron B consistently fires just **before** neuron A



action potential in the postsynaptic cell (B)

action potential in the presynaptic cell (A)



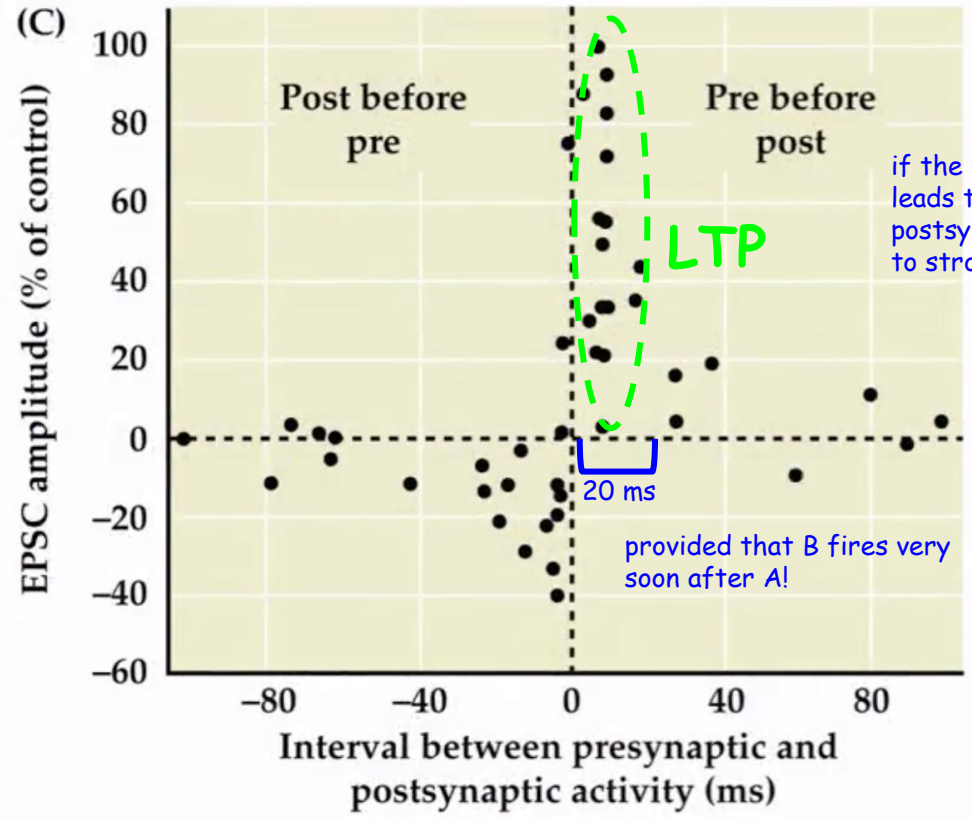
Notice that now there is a **weakening** of the connection between A and B: this is what happens when the postsynaptic neuron is influenced to fire by some other input (not neuron A).

NEUROSCIENCE 5e, Figure 8.18 (Part 2)

© 2012 Sinauer Associates, Inc.

Credit: L. White (Duke, coursera.org)

STDP: Spike-Timing Dependent Plasticity



if the presynaptic input leads to the firing of the postsynaptic cell, this may lead to strong Long-Term Potentiation…

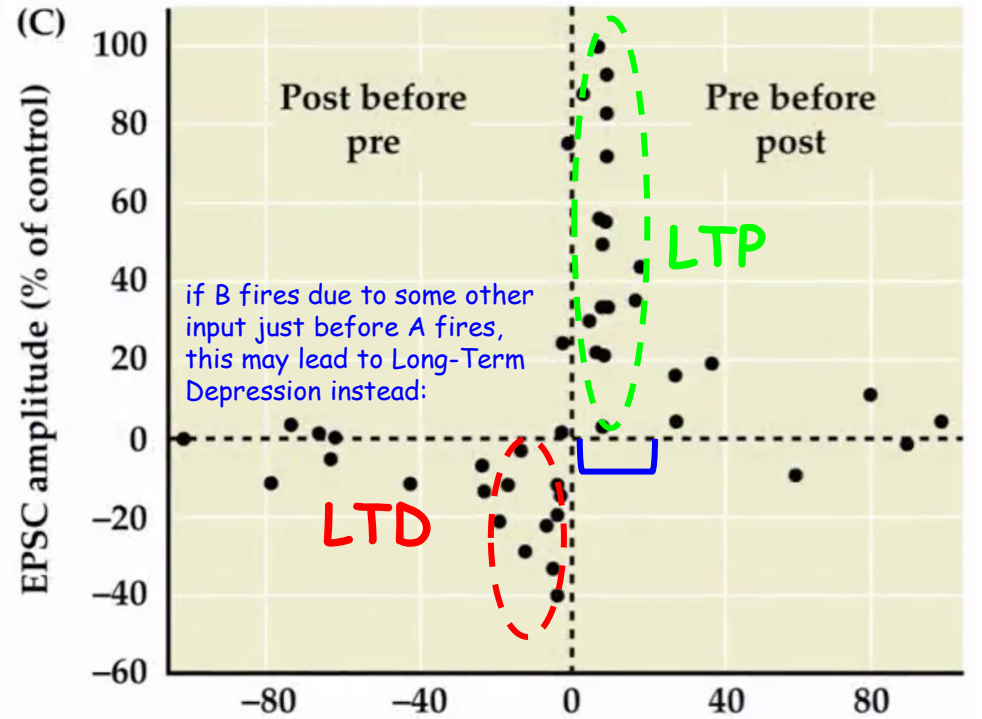
provided that B fires very soon after A!

NEUROSCIENCE 5e, Figure 8.18 (Part 3)

© 2012 Sinauer Associates, Inc.

Credit: L. White (Duke, coursera.org)

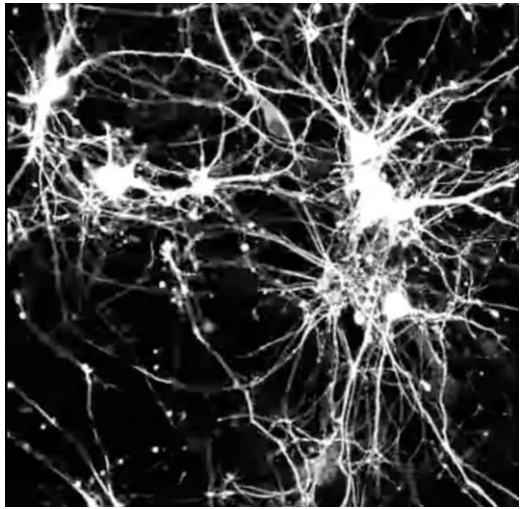
STDP: Spike-Timing Dependent Plasticity



Note: in order for LTD to happen, the synapse has to be used, but out of synchrony (a "near miss").
If it's not used, it may stay unchanged!

NEUROSCIENCE 5e, Figure 8.18 (Part 3)
© 2012 Sinauer Associates, Inc.

Credit: L. White (Duke, coursera.org)



https://www.youtube.com/watch?v=vZ1P1CVfCNc

A brain cell reaching out for connections #science

How can all this help us learn?

- What we just saw showed that structured inputs (coordinated in time) lead to strengthening (Long Term Potentiation) of connections
- And that "noisy" uncoordinated activity patterns lead to weakening (Long Term Depression)
- So our brain imposes a temporal filter on its own activity patterns, as a way of selecting those that lead to an increase in coordination of firing across synaptic connections.
- One way to impose this is through interactions with the environment!

Source: L. White (Duke, coursera.org)

How can all this help us learn?

- In a developing brain, **sensory experience** might be one source of structure that favor the potentiation of synaptic connections;
- whereas the **noise** (irrelevant structure) may lead to the "**pruning**" of synaptic connections that are not functional.

- *Neurons communicate using both electrical and chemical signals.*
- *Life experiences change the nervous system.*

BrainFacts.org

AI that supplies answers too early prevents from:

- experimenting
- forming hypotheses
- generating their own predictions
- discovering patterns

Source: L. White (Duke, coursera.org)

Hebb's Postulate

- One of the first people to think about the plastic nature of the brain was Canadian psychologist Donald Hebb (1904-1985).
- In *The Organization of Behavior* (1949), he proposed:
  - **coordinated** activity of a presynaptic terminal and a postsynaptic cell would **strengthen** the synaptic connection between them
  - conversely, **uncoordinated** activity between synaptic partners would **weaken** their synaptic connections
  - "neurons that fire together, wire together"
  - "out of sync, lose the link"

- experimentally proven to be informative in understanding the cellular mechanisms of synaptic change only many years later, in the 70s and 80s

Types of Learning

- Reinforcement learning
  - Learning through rewards in uncertain environments

- Unsupervised learning
  - Learn statistical regularities and build internal categories automatically i.e. faces, voices stand out
  - Dimensionality reduction (PCA)
  - Higher brain areas predict what lower areas should see
  - Modern unsupervised ML has shifted toward **self-supervised learning (SSL)**: the model predicts missing parts of its own input.

- Supervised learning
  - Learn from training data, labels.

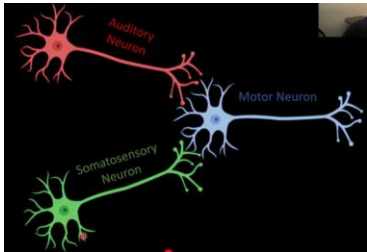
Summary – how biological neurons learn

- “Neurons that fire together, wire together”
- "Out of sync, lose the link"

We can now restate it in more technical terms:

- changes in synaptic strength (consistent with Hebb's postulate) are mediated by long-term potentiation (LTP) and long-term depression (LTD)
- spike-timing dependent plasticity (STDP) explains how synaptic change can occur one spike at a time under physiological conditions in neural networks

https://www.youtube.com/watch?v=-wakk5Iwqj8



If you would like to learn more:

Many great resources online!

https://pll.harvard.edu/series/fundamentals-neuroscience

https://www.coursera.org/learn/medical-neuroscience/

Beautiful video about how synapses work:
https://youtu.be/OvVI8rOEncE?t=69

Next time: how to improve the perceptron?

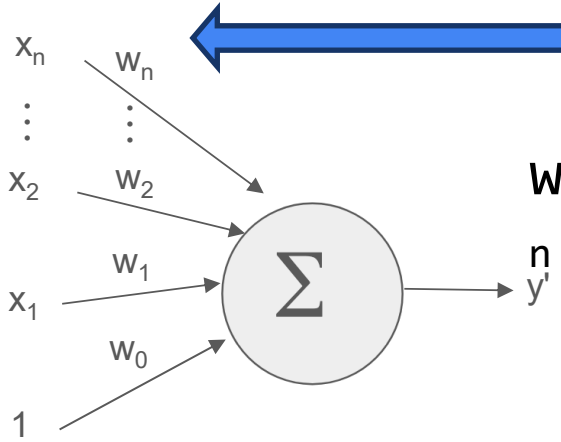
Connecting multiple neurons together…

Session 4: Multi-layer perceptrons
- Introduction to feedforward networks (simple neural networks)
- Loss functions and optimization
- Stochastic Gradient Descent
- Backpropagation

How does that work mathematically?

- labels are 1 or -1
- calculate difference label - prediction $y_n - y_n\hat{}$
- y' depends on w
- reduce w if y' is too large
- increase if w' is too small

$$y_n - y_n\hat{} = \begin{cases} 0, & y_n = yn\hat{} \\ 2, & y_n = 1,\ y_n\hat{} = -1 \\ -2, & y_n = -1,\ y_n\hat{} = 1 \end{cases}$$



$$w = w + \alpha(y_n - y_n\hat{})x$$

Q: if y-y^ = 0, what happens to w?

Q: if y-y^ = 2, what happens to w?

Q: if y-y^ = -2, what happens to w?